

Министерство цифрового развития
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Сибирский государственный университет телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Отчёт

по лабораторной работе № 1 «Первичный анализ и предобработка данных»

Выполнил:
студент группы ИП-413
Рябец И. А.

Работу проверил: _____
должность преподавателя _____
ФИО преподавателя

Новосибирск 2026 г.

Введение (задание)

В качестве набора данных выбран датасет House Price (Kaggle: juhishojani/house-price) — объявления о недвижимости в Индии, 187531 объект и 21 признак. Изначально в таблице есть два ценовых поля: price (in rupees) — цена за единицу площади и amount(in rupees) — полная стоимость в записи вида «42 Lac» / «1.40 Cr». В итоговой постановке целевой переменной выбрана полная стоимость amount: её можно однозначно перевести в рупии, и именно её имеет смысл предсказывать как цену объекта. Price за квадрат оставлен только для разведочных графиков и затем удалён, чтобы не было утечки. Цель работы — EDA, предобработка и базовый пайплайн регрессии.

Основная часть

1. Загрузка и первичный осмотр

Данные загружены через kagglehub. Исходная таблица сохранена в df_original, обработка шла на копии df. Имена столбцов приведены к нижнему регистру, target = 'amount(in rupees)'.

```
df_original = pd.read_csv(path + "/house_prices.csv")
df = df_original.copy()
target = 'amount(in rupees)'
df.columns = df.columns.str.lower()
```

Amount нельзя анализировать как число сразу: это строки с индийскими единицами. Написана функция parse_amount: lac → ×100 000, cr → ×10 000 000, иначе попытка float. Неразбираемые значения (в том числе Call for Price) становятся NaN.

```
df['amount(in rupees)'] = df['amount(in rupees)'].str.lower()
df['amount(in rupees)'] = df['amount(in rupees)'].apply(parse_amount)
```

После парсинга 9684 значения — NaN, это ровно строки Call for Price. Размер набора 187531×21. Числовые типы: index, price (in rupees), dimensions, plot area; после парсинга amount тоже число. Dimensions и Plot Area полностью пустые. Price имеет пропуски (17665). Первые строки подтверждают смешанный формат площадей, этажей и парковки.

2. Анализ пропусков

```
print(df.isna().sum(axis=0))  
sns.heatmap(df.isna())
```

Наибольшие доли пропусков до фильтра:

plot area / dimensions	100%
society	58.5%
super area	57.4%
car parking	55.1%
overlooking	43.4%
carpet area	43.0%
facing	37.5%
ownership	34.9%
balcony	26.1%
price (in rupees)	9.4%

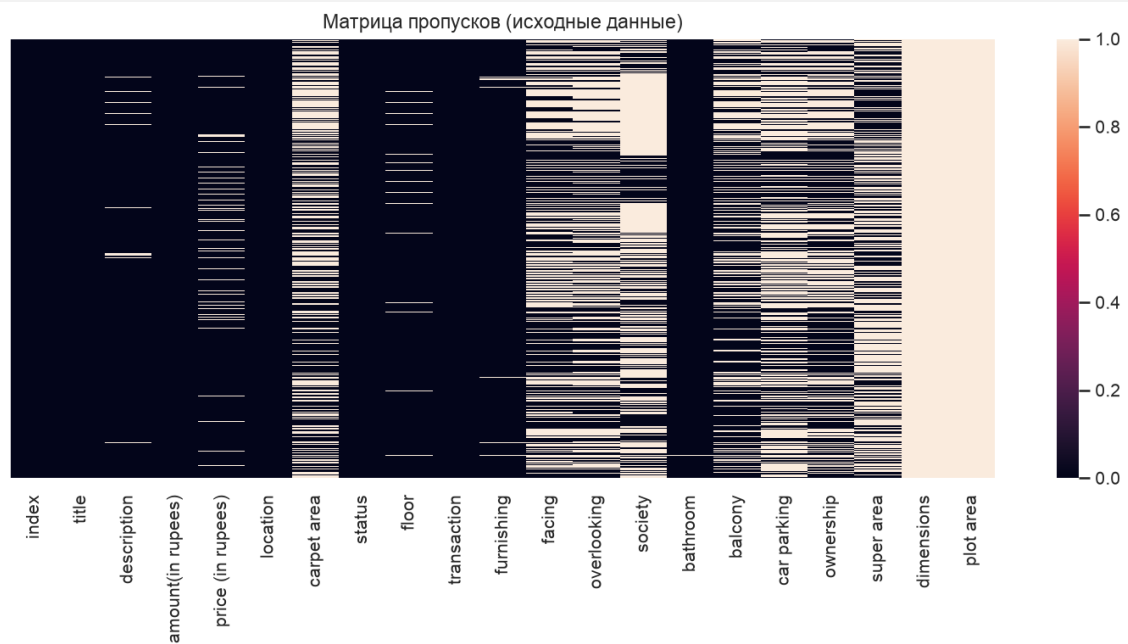


Рисунок 1 — Тепловая карта пропусков

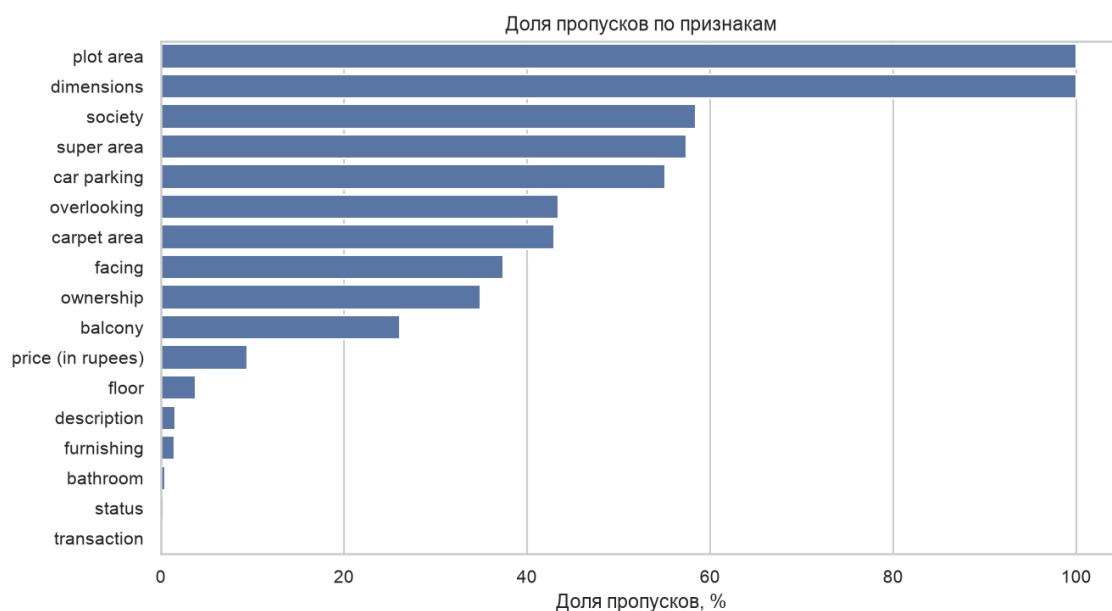


Рисунок 2 — Доля пропусков по признакам

Стратегия, сформулированная по карте. Dimensions и plot area удаляются сразу: они абсолютно пустые. Overlooking и ownership слишком дырявые, из остальных полей их не восстановить. Description и society — высокая кардинальность, для модели «как есть» бесполезны. Title и index тоже не несут обобщаемой информации. Status почти константа Ready to Move. Car parking сначала казался кандидатом на удаление из-за 55% пропусков, но затем из него удалось вытащить два рабочих признака. Carpet area не удалялся: позже выяснилось, что он дополняет super area, а не дублирует его.

3. Анализ числовых признаков

Для amount после парсинга: среднее ≈ 12.0 млн, медиана 7.8 млн, СКО ≈ 39.4 млн, асимметрия ≈ 270 . Максимум доходит до 1.4×10^{10} — хвост ломает все статистики.

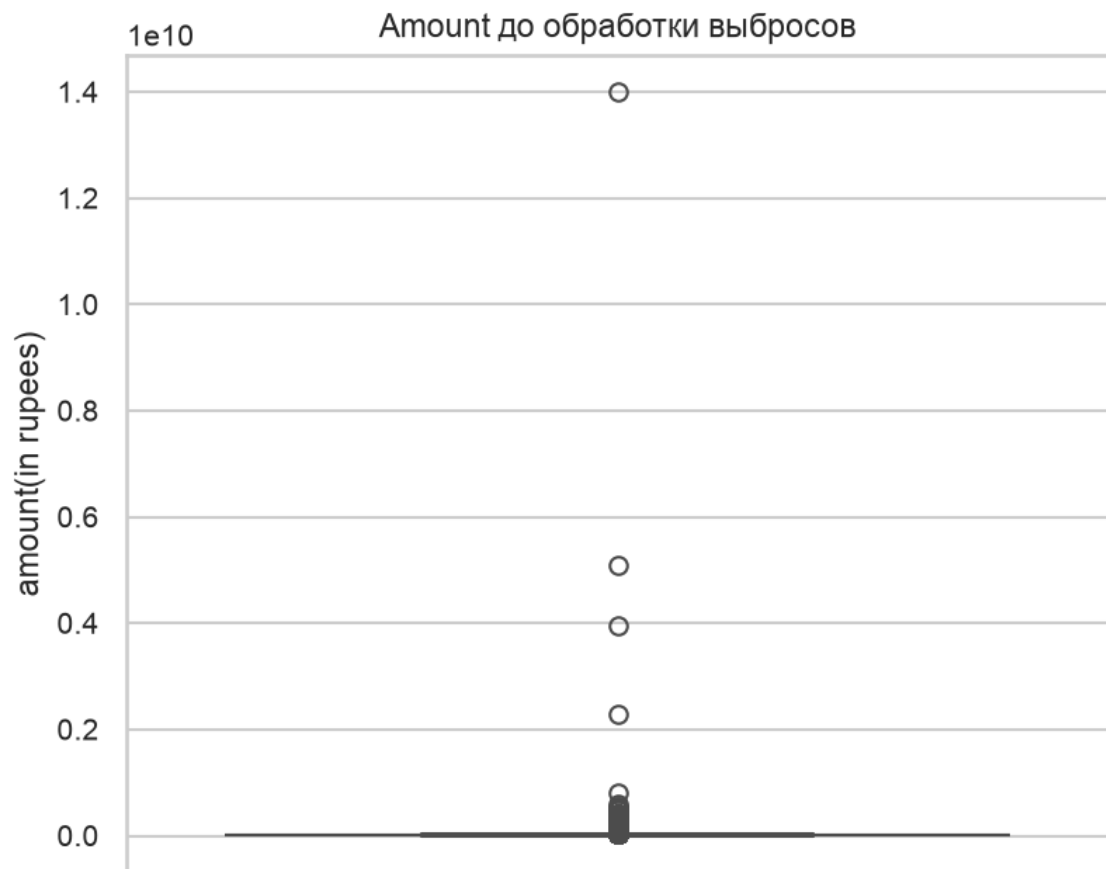


Рисунок 3 — Boxplot amount до обработки выбросов

Выбросы в таргете отсечены по 99-му процентилю: объекты дороже порога удаляются, цена остальных не подменяется. Для целевой переменной это честнее цензурирования. $q_{99} = 70\,000\,000$ рублей. После фильтра 176665 строк; среднее ≈ 11.19 млн, медиана 7.7 млн, СКО ≈ 10.29 млн, skew ≈ 2.37 . Пропуски amount при этом исчезают: NaN не проходит условие $\leq q_{99}$, вместе с хвостом уходят и Call for Price.

```
q99 = df[target].quantile(0.99)
df = df[df[target] <= q99].copy()
```

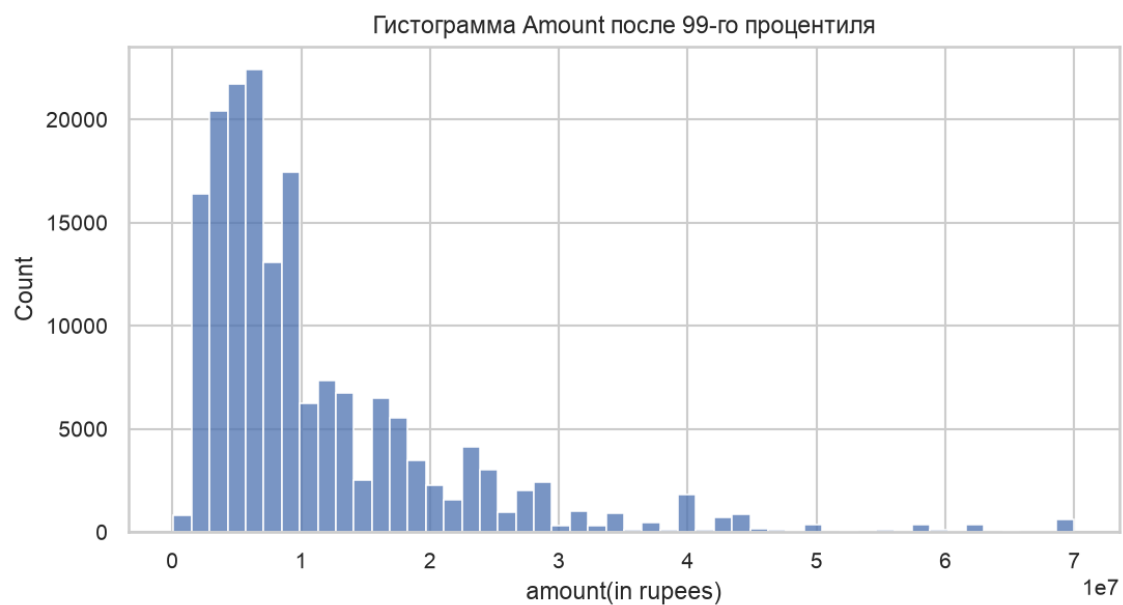


Рисунок 4 — Гистограмма amount после отсечения 99-го перцентиля

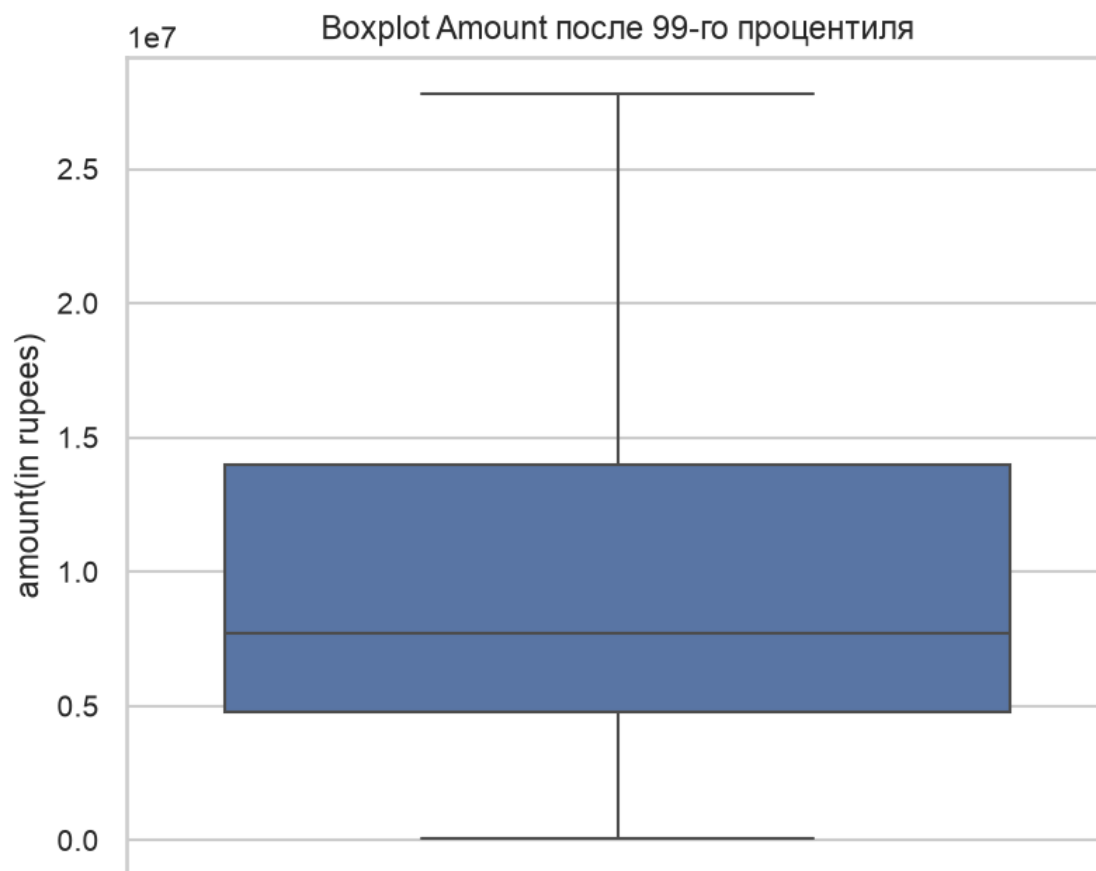


Рисунок 5 — Boxplot amount после отсечения

4. Анализ категориальных признаков

Status практически всегда Ready to Move, countplot строился по выборке 20 000 строк. Признак почти константа и позже удалён. Description, society, title — высокая кардинальность. Location: 81 значение — высокая, но рабочая кардинальность. One-hot «в лоб» раздувает таблицу, а max_categories=20 отбрасывает мелкие города в одну корзину. Поэтому location вынесен в target encoding: каждому городу ставится сглаженное среднее amount по train (smoothing=10). One-hot остаётся для facing, где категорий мало.

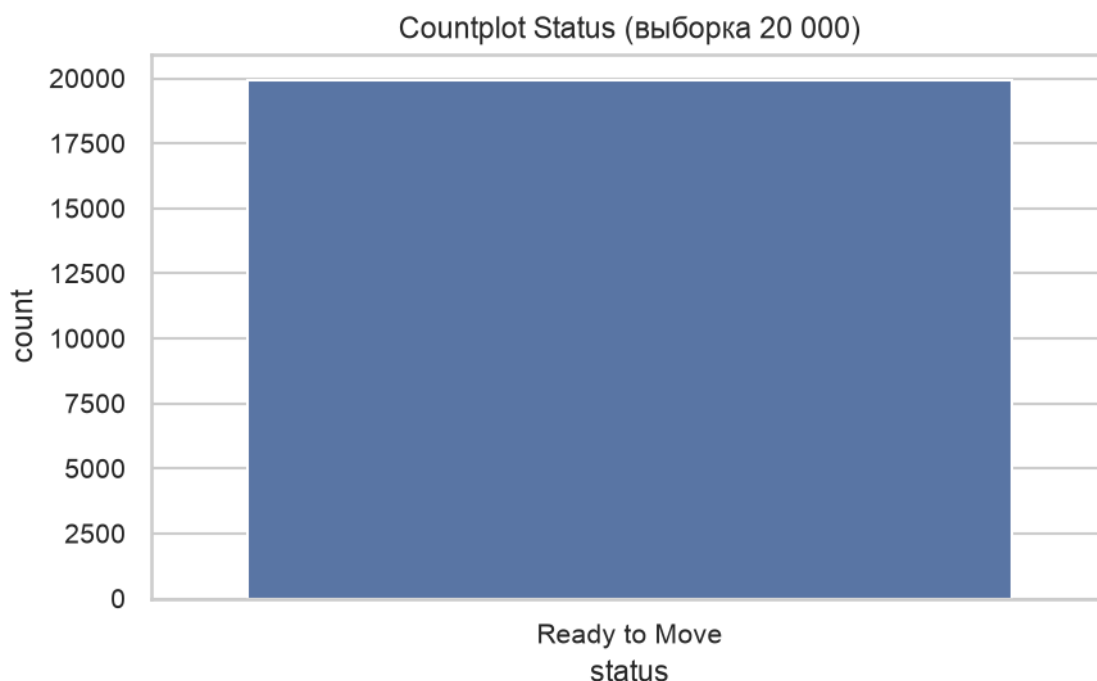


Рисунок 6 — Countplot Status

Bathroom и balcony — строки-счётчики («1»...«10», «> 10»). Больше ванн логически связано с более дорогим объектом, поэтому они переведены в числа, «> 10» → 11. Пропуски (759 у ванн, 47045 у балконов) не заполнялись вручную: медиана ставится уже в пайплайне на train, без утечки.

```
df['bathroom'] = df['bathroom'].replace('> 10', 11)
df['bathroom'] = pd.to_numeric(df['bathroom'], errors='coerce')
df['balcony'] = df['balcony'].replace('> 10', 11)
df['balcony'] = pd.to_numeric(df['balcony'], errors='coerce')
```

Furnishing — порядковый признак (нет мебели → частично → полностью), кодируется 0/1/2. Facing оставляется категорией: пропуски =

'missing', строки в нижний регистр. Floor в сыром виде («10 out of 11», «Ground out of 7») сначала казался слишком грязным, но позже из него вытащили числа этажа — это дало один из самых сильных приростов модели.

5. Анализ взаимосвязей

Scatterplot amount–bathroom: дешёвые объекты чаще с 1–2 ванными, 4+ ванных сдвигаются вправо по цене.

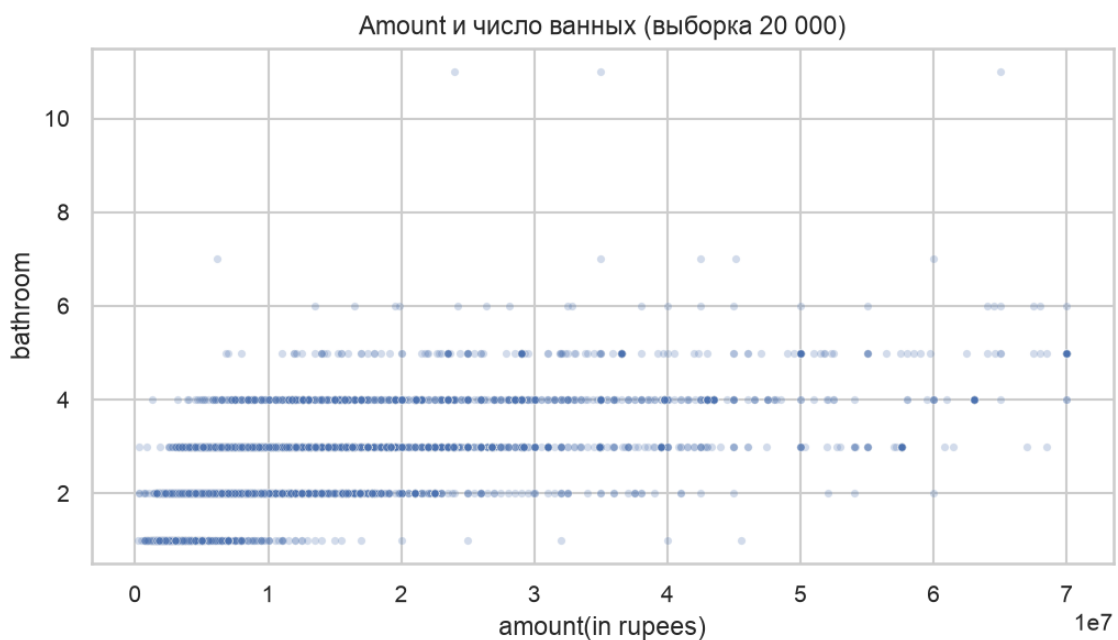


Рисунок 7 — Amount и число ванных

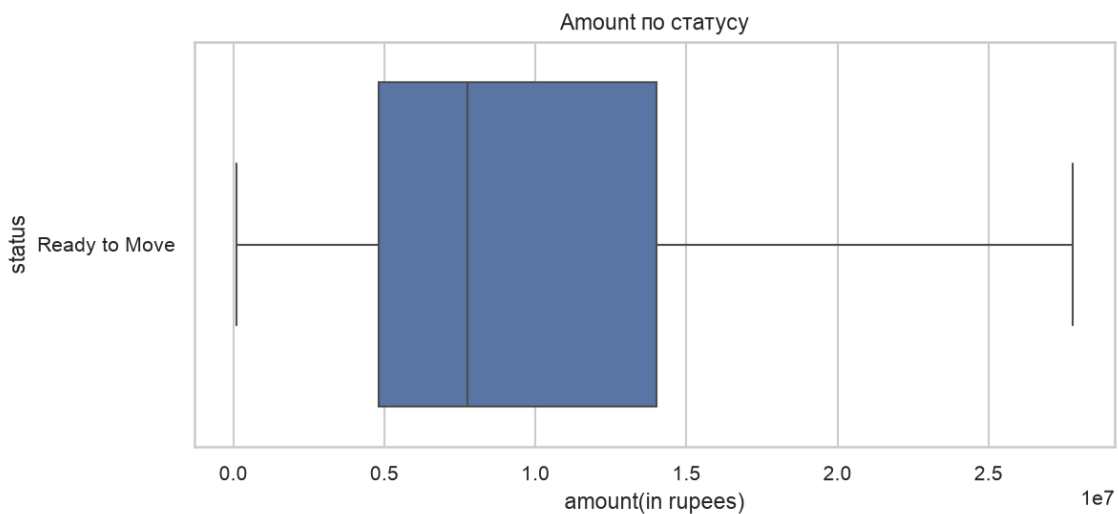


Рисунок 8 — Amount по статусу

По статусу сравнивать почти нечего — доминирует Ready to Move.

```
df = df.drop(['status', 'index', 'title', 'price (in rupees)', 'description',  
            'overlooking', 'society', 'plot area', 'dimensions', 'ownership'], axis=1)
```

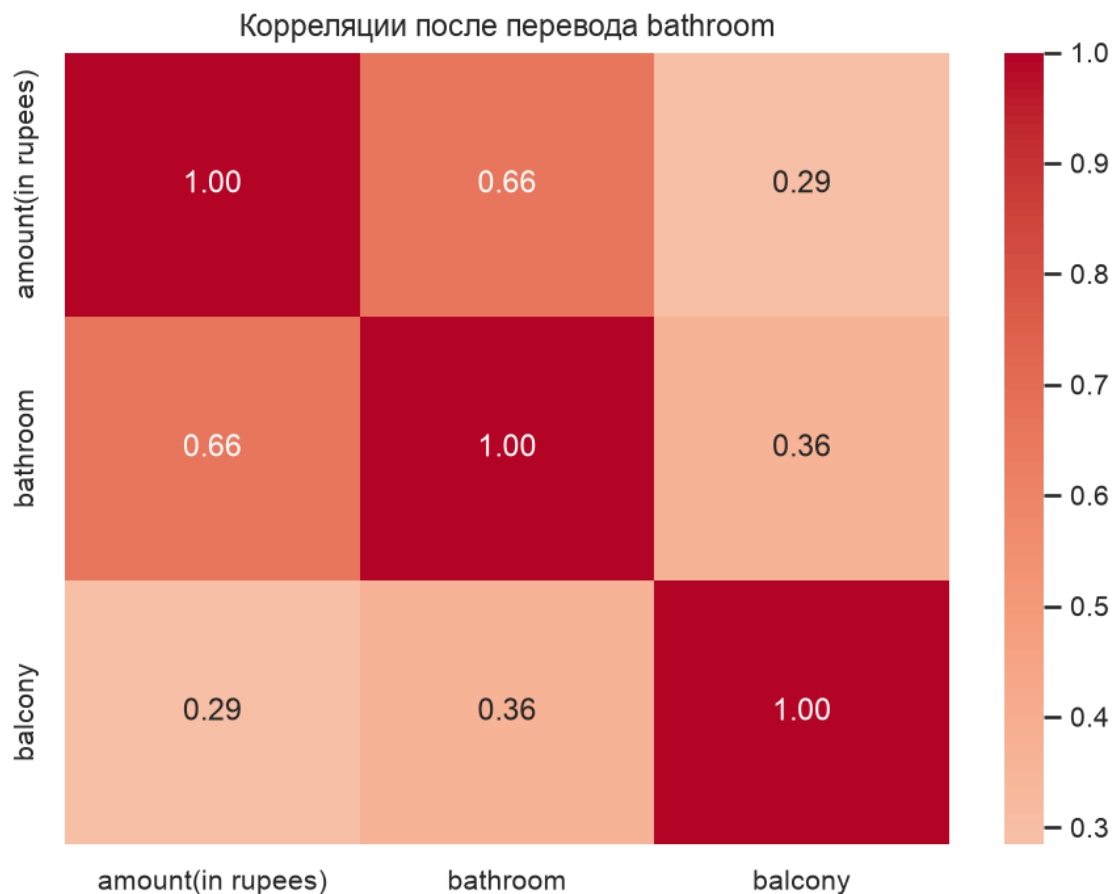


Рисунок 9 — Корреляции числовых полей после перевода bathroom

Парковка. Гипотеза: число мест и крытость повышают стоимость; NaN = нет парковки. Из строки сделаны car covered (1, если есть covered) и parking count (число записей по запятым, иначе 0).

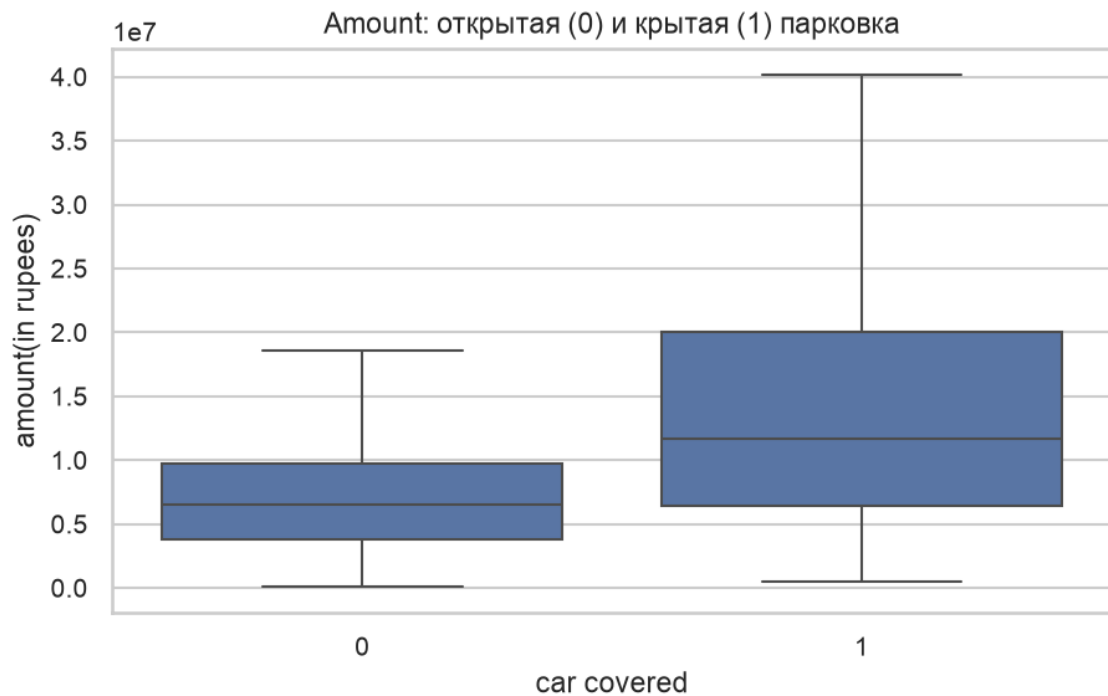


Рисунок 10 — Amount при открытой и крытой парковке

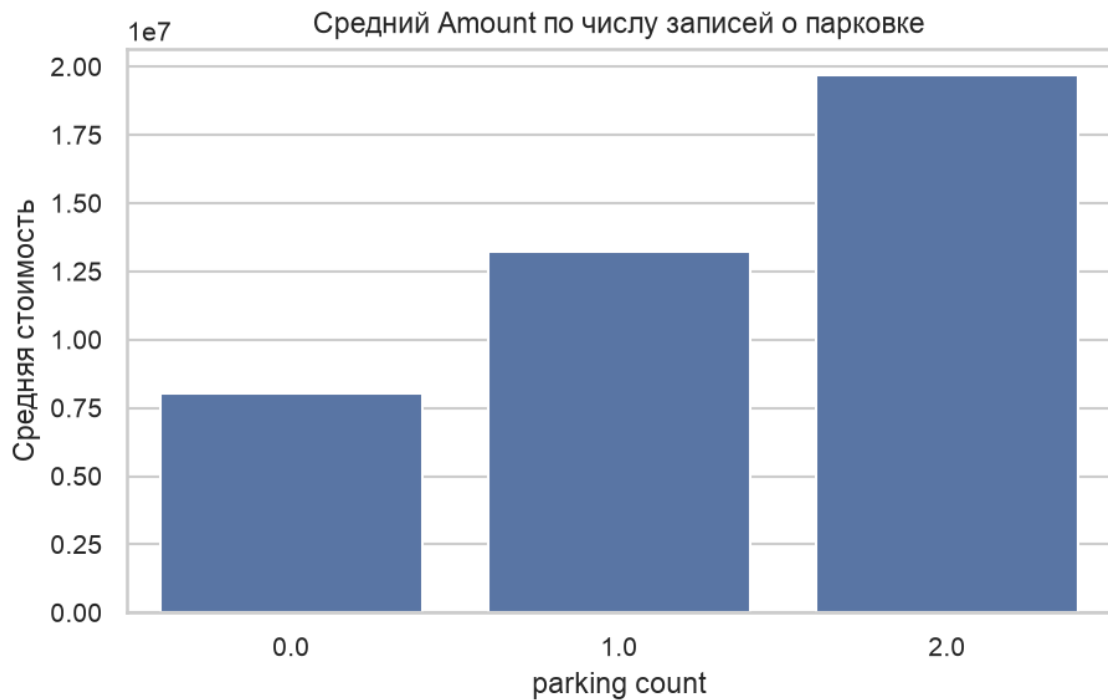


Рисунок 11 — Средний amount по числу записей о парковке

Гипотеза подтвердилась: без крытой парковки средняя цена ≈ 8.46 млн, с крытой ≈ 15.40 млн. 0 / 1 / 2 записи о парковке: ≈ 8.05 / 13.24 / 19.68 млн. Признаки оставлены; сырой car parking потом удалён.

Меблировка и тип сделки. Средние amount: Unfurnished ≈ 10.09 млн, Semi-Furnished ≈ 12.38 млн, Furnished ≈ 10.61 млн. Полностью меблированная дешевле полумеблированной — на первый взгляд странно. Идея: fully furnished чаще уже жилые resale, а resale дешевле новостройки.

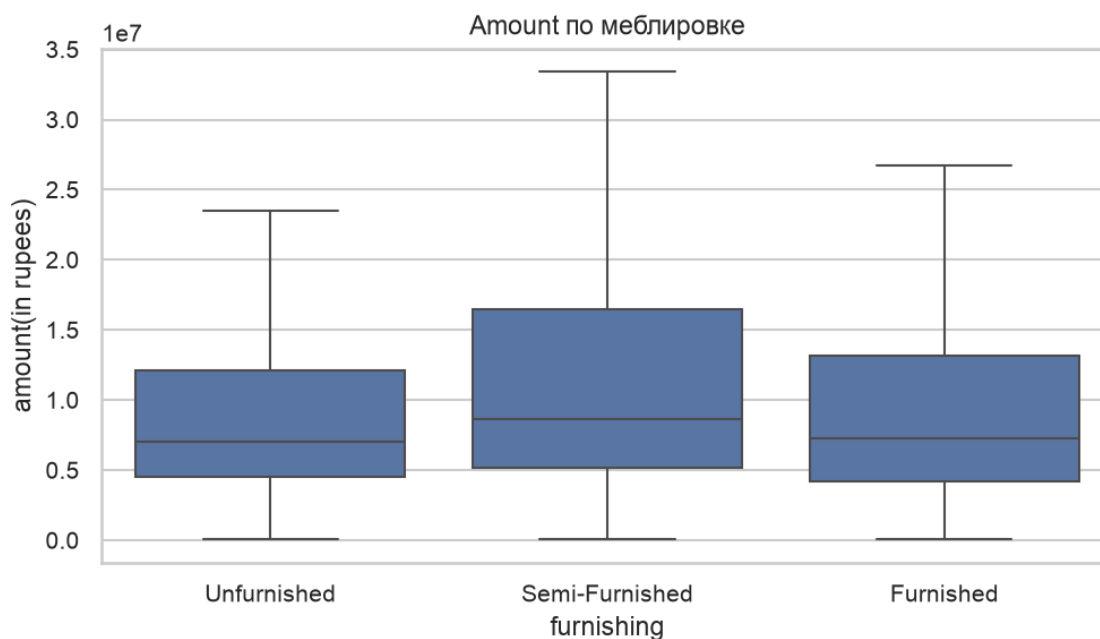


Рисунок 12 — Amount по степени мебелировки

Доля resale: Unfurnished 73%, Semi-Furnished 77%, Furnished 86%. New Property в среднем ≈ 13.25 млн, Resale ≈ 10.61 млн (примерно на 20% дешевле). Semi дороже furnished примерно на 1.8 млн. Сдвиг furnished в сторону resale объясняет часть разрыва, но не весь: разница сделок не перебивает разницу мебелировки полностью, гипотеза подтверждена частично. Редкие Other и Rent/Lease отброшены. Transaction закодирован бинарно: New Property = 1, Resale = 0.

Площади. Super area и carpet area почти не пересекаются: если super заполнен, carpet пуст всегда; если super пуст, carpet отсутствует лишь в 0.09% строк. Это не два независимых пропуска, а два способа записать площадь. Поэтому оба поля парсятся в sqft и оба остаются: для части объектов работает одно, для части — другое.

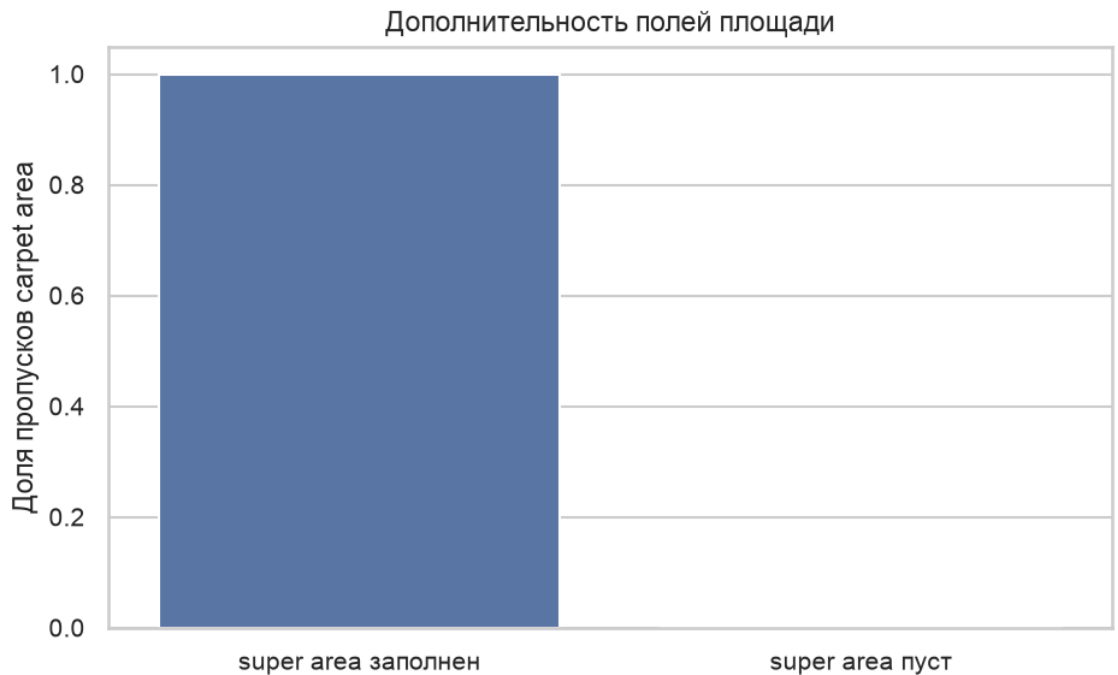


Рисунок 13 — Дополнительность *super area* и *carpet area*

Парсер `parse_area` вытаскивает число и единицу: `sqft` как есть, `sqm`×10.76, `acre`×43560, `sqyrd`×9, `sqin`/144. После перевода в хвостах оказались невозможные площади (вплоть до десятков миллионов `sqft`). Оставлены `carpet` в [100, 20000] или `NaN` и `super` в [100, 5000] или `NaN`. Строк стало 174824. Сырые текстовые площади и флаги `missing` удалены: после парсинга флаги дублируют `NaN` в числовых столбцах.

Итоговая корреляция с `amount` (после всех преобразований):

<code>carpet_area_sqft</code>	0.70
<code>bathroom</code>	0.66
<code>super_area_sqft</code>	0.64
<code>parking count</code>	0.38
<code>car covered</code>	0.33
<code>balcony</code>	0.28
<code>transaction</code>	0.11
<code>furnishing</code>	0.08

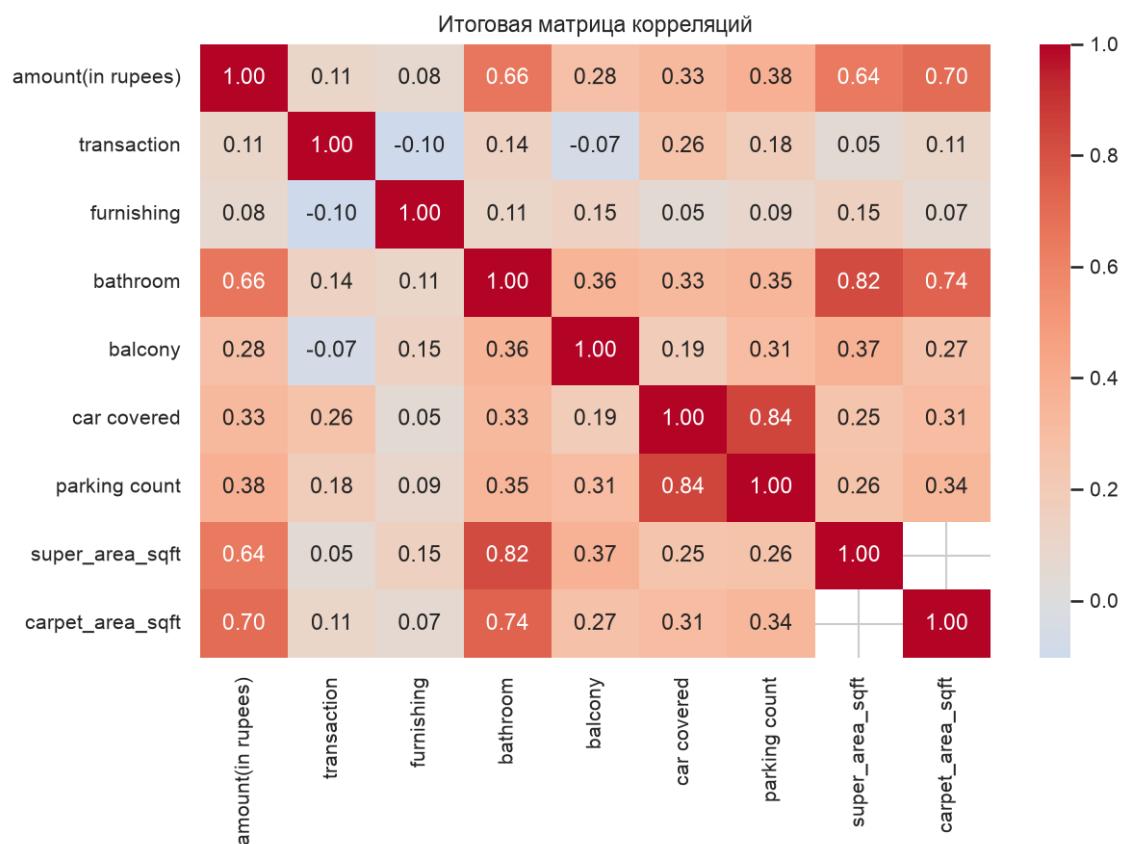


Рисунок 14 — Итоговая матрица корреляций

Самые сильные линейные связи с таргетом — площади и число ванн. Furnishing слабый, как и следовало из немонотонных средних. Car covered и parking count связаны между собой, оба из одного исходного поля.

6. Базовая предобработка

- 1) Копия датафрейма, нижний регистр имён, target = amount.
- 2) Парсинг amount из lac/cr; Call for Price → NaN.
- 3) Удаление объектов выше 99-го перцентиля amount (порог 70 млн).
- 4) Удаление status, index, title, price, description, overlooking, society, plot area, dimensions, ownership, затем сырых car parking, floor, текстовых площадей.
- 5) Bathroom и balcony → числа, «> 10» → 11; импутация только в пайплайне.

- 6) Из парковки: car covered и parking count.
- 7) Furnishing 0/1/2; transaction 1/0; только New Property и Resale.
- 8) Парсинг площадей в sqft и отсеечение невозможных значений; facing = missing при пропуске.
- 9) Location кодируется TargetEncoder на train и переносится на test.
- 10) Floor парсится в floor_current и floor_total; Ground = 0; битый формат без total отбрасывается.
- 11) Модель учится на $\log_{1p}(\text{amount})$; финальный алгоритм — RandomForest с гиперпараметрами Optuna (50 trial по MAPE). Сравнивались также LightGBM и CatBoost.

Итоговые признаки текущей версии: location (target encoding), transaction, furnishing, facing, bathroom, balcony, car covered, parking count, super_area_sqft, carpet_area_sqft, floor_current, floor_total.

7. Обработка выбросов

Основной выброс — amount. Метод: удаление верхнего 1%, не винзоризация. Дополнительно отфильтрованы нереалистичные площади после парсинга единиц: иначе один acre, записанный ошибкой, давал десятки миллионов sqft и ломал и корреляции, и дерево. Сравнение рисунков 3 и 5 показывает, что после отсеечения цен масштаб оси читаемый, асимметрия падает с ~270 до ~2.4.

8. Базовый пайплайн и эксперименты

Пайплайн фиксирован: train/test 80/20, random_state=42, числа — median+scaler, facing — one-hot (у CatBoost location и facing шли как нативные категории). Сначала менялись признаки и масштаб таргета при RF(n_estimators=100), затем подобрали гиперпараметры RF, после этого на том

же препроцессинге сравнили LightGBM и CatBoost. Ход прогонов сведён в таблицу 1.

Кодировка location. One-hot по топ-20 городам (эксп. 1) дал MAE 950 тыс. и 18 минут обучения. TargetEncoder со сглаживанием 10 (эксп. 2) снизил MAE до 891 тыс., MAPE до 11.21% и ускорил обучение до ~46 с: вместо пачки dummy — одно число, средний уровень рынка города.

Склейка площадей. Поскольку carpet и super почти не пересекаются, пробовали одну колонку area_sqft: сначала carpet.fillna(super) (эксп. 3), затем с поправкой на отношение медиан, потому что super обычно больше ковра (эксп. 4). Оба варианта хуже двух отдельных колонок: MAE 918 и 899 тыс. против 891 тыс. Флаг area_missing при сохранённых двух площадях (эксп. 5) почти ничего не дал (важность 0.00015). Вывод: carpet и super — разная семантика, дерево само разбирается с NaN через медиану в пайплайне.

Этаж. Строка «10 out of 11» разобрана на floor_current и floor_total, Ground → 0 (эксп. 6). Это лучший прирост среди признаков: MAE 823 тыс., R^2 0.9426, MAPE 10.14%. Сжатие в floor_ratio = current/(total+1) (эксп. 7) часть сигнала теряет (MAE 876 тыс.): высота дома и номер этажа для цены — не одно и то же.

Логарифм таргета. У amount skew ≈ 2.36 , поэтому лес учили на log1p, а метрики считали после expm1 (эксп. 8). MAPE упал до 9.10%, MAE до 813 тыс.; R^2 чуть ниже, чем у линейного таргета с этажами (0.9388 против 0.9426), потому что оптимизация идёт в логарифмической шкале. Флаг последнего этажа top_floor (эксп. 9) добавлен из идеи, что условие current==total дереву неудобно ловить парой чисел. Прирост крошечный (MAE 812.3 тыс., MAPE 9.09%), важность флага 0.003 — гипотеза почти не подтвердилась. В финальной версии top_floor убран.

Подбор гиперпараметров (эксп. 10). Optuna, 50 испытаний, цель — минимум MAPE. Искались n_estimators (50–300), max_depth (5–30),

min_samples_split (2–20), min_samples_leaf (1–10), max_features и max_samples. Лучший trial 46: n_estimators=180, max_depth=25, min_samples_split=2, min_samples_leaf=1, max_features≈0.751, max_samples≈0.956. На тесте: MAE 799 909, RMSE 2 517 780, R^2 0.9414, MAPE 8.93%. Глубокие деревья без сильного прунинга оказались лучше мелких; 180 деревьев чуть сильнее дефолтных 100. Оговорка: objective смотрел MAPE на том же test, поэтому цифра чуть оптимистичнее, чем при отдельном validation/CV.

Сравнение моделей (эксп. 11–13). На тех же признаках и log-таргете прогнаны градиентные бустинги. LightGBM с широкими деревьями «из коробки» (2000 деревьев, lr=0.01, num_leaves=256) дал MAE 886 тыс. и MAPE 9.76% — хуже тюнингованного RF, хотя R^2 уже 0.9412. Optuna, 20 испытаний по MAPE (эксп. 12), вывела n_estimators=1971, learning_rate≈0.089, num_leaves=136, max_depth=10. После переобучения: MAE 841 123, RMSE 2 515 384, R^2 0.9416, MAPE 9.39% (лучший trial на тесте был 9.36%). R^2 чуть выше, чем у RF, но MAE и MAPE заметно хуже: бустинг лучше сажает крупные ошибки, лес точнее в относительной шкале. Первый прогон Optuna для LGBM упал с ошибкой: subsample/colsample/reg_lambda случайно назвали тем же именем, что min_child_samples.

CatBoost (эксп. 13) брал location и facing как cat_features, без TargetEncoder и one-hot: iterations=2000, lr=0.05, depth=8, early_stopping на test. Обучение ~4.3 мин, bestIteration=1999 (стоп не сработал). MAE 928 719, R^2 0.9380, MAPE 10.29% — хуже и RF, и LGBM. Нативные категории не отыграли отставание: без отдельного тюнинга и при сыром location модель слабее, чем RF с target encoding.

Итог сравнения. В финал берётся Random Forest после Optuna: лучшие MAE и MAPE, интерпретируемые важности, весь препроцессинг уже заточен

под sklearn-пайплайн. LightGBM оставляем как близкую альтернативу по R^2 , CatBoost — нет.

№	Идея	Изменения	MAE	RMSE	R^2	MAPE	Обучение
1	Baseline: location как категория	OneHotEncoder (max_categories=20); RF, 100 деревьев	950 462	2 672 729	0.9323	12.71%	1093 с
2	Target encoding location	TargetEncoder(smoothing=10) на train; one-hot только для facing	891 335	2 598 240	0.9360	11.21%	45.7 с
3	Одна площадь вместо двух	area_sqft = carpet.fillna(super); отдельные carpet/super удалены	918 338	2 652 981	0.9333	11.72%	46.5 с
4	Склейка площадей через отношение медиан	area_sqft = carpet.fillna(super / (med_super / med_carpet))	899 367	2 623 582	0.9347	11.43%	37.0 с
5	Флаг пропуска площади	Две колонки sqft сохранены + area_missing	890 026	2 592 869	0.9362	11.22%	56.2 с
6	Парсинг этажа в два числа	floor_current и floor_total из «N out of M»; Ground→0	822 873	2 493 350	0.9426	10.14%	60.3 с
7	Одна колонка-отношение этажей	floor_ratio = current / (total+1) вместо двух чисел	875 659	2 623 242	0.9364	10.70%	57.0 с
8	Логарифм таргета	Обучение на log1p(amount), предсказание expm1; этажи current+total	812 812	2 573 831	0.9388	9.10%	59.4 с
9	Флаг последнего этажа	top_floor = (current==total) + log-target + два числа этажа	812 300	2 573 551	0.9388	9.09%	62.2 с
10	Оптимизация гиперпараметры RF	50 trial, min MAPE; n_estimators=180, max_depth=25, min_samples_split=2, min_samples_leaf=1, max_features≈0.75, max_samples≈0.96; log-target, два этажа	799 909	2 517 780	0.9414	8.93%	50 trial

11	LightGBM, базовые параметры	LGBMRegressor: n_estimators=2000, lr=0.01, num_leaves=256, min_child_samples=5; log-target, те же признаки	886 385	2 522 470	0.9412	9.76%	—
12	Optuna: гиперпараметры LightGBM	20 trial, min MAPE; n_estimators=1971, lr≈0.089, num_leaves=136, max_depth=10, min_child_samples=46, subsample≈0.84, colsample≈0.83	841 123	2 515 384	0.9416	9.39%	20 trial
13	CatBoost, нативные категории	iterations=2000, lr=0.05, depth=8; cat_features=location+facing без TE/one-hot; early_stopping на test	928 719	2 591 000	0.9380	10.29%	4.3 мин

Таблица 1 — Журнал экспериментов (test, split 80/20, random_state=42)

Что оставляем. Две площади, target encoding location, два числа этажа, log-target, подобранный Random Forest. Склею площади, floor_ratio, one-hot по 81 городу, top_floor, LightGBM и CatBoost — нет.

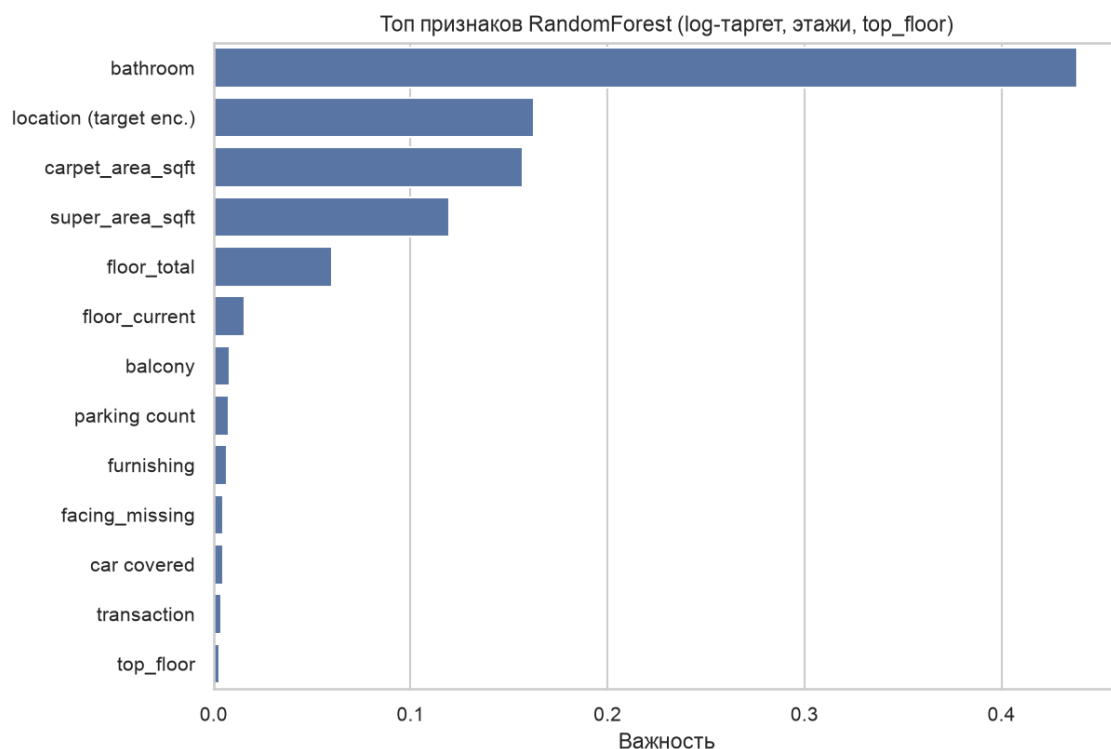


Рисунок 15 — Важности признаков текущей версии (эксп. 9)

На логарифме таргета первым становится bathroom (0.44), затем location (0.16) и две площади. floor_total заметнее floor_current: этажность дома сильнее номера этажа. Парковка, мебель, facing и top_floor — слабый хвост.

Заключение

Проблемы данных: пустые dimensions/plot area; дыры в парковке, facing, площадях; текст вместо чисел в amount, bathroom, balcony, area и floor; Call for Price; хвост цен; высокая кардинальность текстов; complementary-пропуски super/carpet.

Что сработало: amount как таргет; отсечение 99-го перцентиля; target encoding location; оставить carpet и super отдельно; распарсить этаж в два числа; учить модель на log1p; подкрутить RF через Optuna (глубже и чуть больше деревьев). Что нет: one-hot по городам, склейка площадей, отношение этажей, флаг last floor; LightGBM и CatBoost не обошли лес по MAE/MAPE.

Итоговая модель — Random Forest (эксп. 10): MAE 799 909, MAPE 8.93%, R^2 0.9414. Ближайший конкурент — LightGBM после Optuna (эксп. 12): MAPE 9.39%, R^2 0.9416. Лучший R^2 среди признаков проб без смены модели — у этажей без логарифма (эксп. 6, 0.9426). CatBoost с нативными категориями (эксп. 13) отстал: MAPE 10.29%.

Ссылка на датасет: <https://www.kaggle.com/datasets/juhibhojani/house-price>

Ссылка на Google Colab: <https://colab.research.google.com/drive/1n0cDIAZl2VhjmzZdJYDrPjBTb9ECFbwh>

Код программы

Ниже код ноутбука. Строки не изменялись. Ячейки только сгруппированы по смыслу, между блоками даны краткие выводы.

```
# ===== 1. Загрузка библиотек и данных =====
# Вывод: датасет с Kaggle, копия df_original. Нужны category_encoders, optuna, lightgbm и catboost.

pip install kagglehub

!pip install optuna -q

!pip install lightgbm xgboost catboost -q

!pip install category_encoders -q

import kagglehub

# Download latest version
path = kagglehub.dataset_download("juhibhojani/house-price")

print("Path to dataset files:", path)

import os
print(os.listdir(path))

from sklearn.model_selection import train_test_split

import pandas as pd
import numpy as np
import datetime
import matplotlib.pyplot as plt
import seaborn as sns

df_original = pd.read_csv(path + "/house_prices.csv")
df = df_original.copy()

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
```

```

from sklearn.ensemble import RandomForestRegressor

# ===== 2. Парсинг целевой стоимости amount =====
# Вывод: lac/cr переводятся в рупии. Call for Price станет NaN и позже отсеется фильтром по
# квантилю.

print(df.head(5))
target = 'amount(in rupees)'
df.columns = df.columns.str.lower()

# Function для перевода индийских чего-то в рупии
def parse_amount(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip()

    if 'lac' in x:
        return float(x.replace('lac', '').strip()) * 100_000
    elif 'cr' in x:
        return float(x.replace('cr', '').strip()) * 10_000_000
    else:
        try:
            return float(x)
        except ValueError:
            return np.nan

print(df.info())
print(df.describe())
print(df.shape)

print(df.isna().sum(axis=0))

# Пытаюсь узнать есть ли идея в наличии пропусков в таргете, первая мысль это как то связано с
# итоговой ценой
# Экспериментально была выделена интересная закономерность, после фильтрации по 99 квантилю все
# пропуски в таргете исчезают, значит пропуски были как то связаны с выбросами
# df['carpet area'].isna().sum()
df['amount(in rupees)'] = df['amount(in rupees)'].str.lower()
df['amount(in rupees)'] = df['amount(in rupees)'].apply(parse_amount)

# ===== 3. Статистики таргета и отсеечение выбросов =====
# Вывод: skew ~270, нужен 99-й процентиль. После фильтра распределение читаемое, NaN в amount
# исчезают.

Итого – колонки dimensions и plot area удаляются сразу, они абсолютно пусты и бесполезны. Также не
вижу смысла в колонках overlooking, carparking, ownership. Хотя они влияют на таргет, причем
напрямую, но они слишком пусты, а попытке заполнения из уже существующих данных они не поддаются.
Колонки description и society имеют высочайшую координальность, определение логики по ним не имеет
большого смысла, поэтому тоже под удаление. Еще есть полупустая колонка Carpet Area, но ее пока не
трогаю, потому что есть другая почти пустая колонка plot area, потенциально можно пересчитать одно
из другого и заполнить часть пропусков.

#sns.histplot(data=df, x=target, bins=30)
#sns.boxplot(data=df, x='index', y='price (in rupees)')

print(df[target].mean())
print(df[target].median())
print(df[target].std())
print(df[target].skew())

Есть огромные выбросы в целевом параметре, нуждающиеся в цензурировании. Отфильтрую по квантилю
0.99.

q99 = df[target].quantile(0.99)
df = df[df[target] <= q99].copy()
#sns.boxplot(data=df, y=target)

```

```

#sns.histplot(df[target])

# ===== 4. Категории, ванны, удаление лишних столбцов =====
# Вывод: status почти константа; description/society/title не для ML; bathroom — число, пропуски в
# пайплайн;
# price (in rupees) убираем, чтобы не было утечки в таргет amount.

df.info()

Как уже было понятно, Description и society имеют высокую координальность и не применимы для ml.

df['bathroom'] = df['bathroom'].replace('> 10', 11)
df['bathroom'] = pd.to_numeric(df['bathroom'], errors='coerce')
print(df['bathroom'].isna().sum())
print(df['bathroom'].unique())
# Пустоты заполним медианой в пайплайне

#sns.scatterplot(data=df, x=target, y='bathroom', alpha=0.9)

sns.boxplot(data=df, x=target, y='status')

print(df.columns)
df = df.drop(['status', 'index', 'title', 'price (in rupees)', 'description', 'overlooking',
'society', 'plot area', 'dimensions', 'ownership'], axis=1)

print(df.columns)

# ===== 5. Балконы и признаки парковки =====
# Вывод: из car parking получены car covered и parking count. Крытая парковка и больше мест — выше
# amount.

#sns.heatmap(df.select_dtypes(include='number').corr())

df['balcony'] = df['balcony'].replace('> 10', 11)
df['balcony'] = pd.to_numeric(df['balcony'], errors='coerce')
print(df['balcony'].isna().sum())
print(df['balcony'].unique())
# Заполним и переведем в инт на моменте пайплайна

df['car parking'].unique()
# Несколько гипотез, во первых наличие парковки повышает стоимость прямопропорционально количеству
# мест, поэтому имеет смысл в разделить номера парковочных мест, которые ничего не значат от статуса.
# По умолчанию считать что nan это отсутствие парковки, и сделать 2 колонки это количество мест на
# парковке и крытая она или нет. Количество мест обычная целочисленная колонка, крытая или нет колнка
# covered с 1-крытая и 0-открытая

df['car parking'] = df['car parking'].str.lower().str.strip()
df['car covered'] = df['car parking'].str.contains('covered', na=False).astype(int)
df['parking count'] = df['car parking'].str.count(',') + 1
print(df['parking count'].isna().sum())
df['parking count'] = df['parking count'].fillna(0)
print(df.groupby(df['car covered'])[target].mean())

# Гипотеза подтвердилась

Провел головадумать и вывел 2 новых признака. после проверки подтвердил прямую корреляцию между
таргетом и количеством мест а также наличием крытой парковки.

#sns.histplot(df[df_original['Price (in rupees)'].isna()]['amount(in rupees)'])

print(df['amount(in rupees)'].isna().sum())
print(df[target].isna().sum())

# ===== 6. Меблировка и тип сделки =====
# Вывод: fully furnished дешевле semi; чаще resale. New Property кодируется как 1.

```

```

df.info()

print(df['furnishing'].unique())

furnishing_map = {
    'Unfurnished': 0,
    'Semi-Furnished': 1,
    'Furnished': 2
}

df['furnishing'] = df['furnishing'].map(furnishing_map)

print(df.groupby('furnishing')[target].agg(['mean', 'median', 'count']))

#Получается что меблированная стоит дешевле чем полу меблированная. Звучит как бред, хотя на самом
деле есть идея, что меблированная дешевле, потому что квартиры полностью с мебелью уже были жилыми,
то есть resale в параметре Transaction. А это может влиять на цену, то есть полу меблированная стоит
дороже, а полностью меблированная хоть и получает в стоимости, но эта разница покрывается в минус за
счет того что большинство меблированных это resale.

print('\n', pd.crosstab(df['furnishing'], df['transaction'], normalize='index'))

print('\n', df.groupby('transaction')[target].mean())

print(df['transaction'].value_counts())

df = df[df['transaction'].isin(['New Property', 'Resale'])]

df['transaction'] = np.where(df['transaction'] == 'New Property', 1, 0)

# ===== 7. Заготовка ручного CV-target encoding =====
# Вывод: в прогоне использован category_encoders.TargetEncoder, не эта функция.

from sklearn.model_selection import KFold

def target_encoding_cv(train, test, col, n_splits=5, smoothing=10):

    kf = KFold(n_splits=n_splits, shuffle=True, random_state=42)
    train_encoded = pd.Series(index=train.index, dtype = float)

    for train_idx, val_idx in kf.split(train):
        means = train.iloc[train_idx].groupby(col)[target].agg(['mean', 'count'])

        means['encoded'] = (
            (means['count'] * means['mean'] + smoothing - prior) / (means['count'] + smoothing)
        )
        train_encoded.iloc[val_idx] = (
            train.iloc[val_idx][col].map(means['encoded']).fillna(prior)
        )

    means_full = train.groupby(col)[target].agg(['mean', 'count'])
    means_full['encoded'] = (
        (means_full['count'] * means_full['mean'] + smoothing * prior) /
        (means_full['count'] + smoothing)
    )
    test_encoded = test[col].map(means_full['encoded']).fillna(prior)

    return train_encoded, test_encoded

# ===== 8. Площади: дополнительность, парсинг единиц, фильтр хвостов =====
# Вывод: super и carpet почти не пересекаются – оба поля нужны отдельно, без склейки.

print(df['super area'].isna().sum())

```

```

print(df['carpet area'].isna().sum())

df['super_area_missing'] = df['super area'].isna()
df['carpet_area_missing'] = df['carpet area'].isna()
print(df.groupby('super_area_missing')['carpet_area_missing'].agg(['mean', 'count']))

import re

def parse_area(x):
    if pd.isna(x):
        return np.nan

    x = str(x).strip().lower()

    match = re.search(r'([\d.]+\s*([a-z]+)', x)
    if not match:

        try:
            return float(x)
        except ValueError:
            return np.nan

    value = float(match.group(1))
    unit = match.group(2)

    if unit in ['sqft', 'sq.ft', 'sq-ft']:
        return value
    elif unit == 'sqm':
        return value * 10.7639
    elif unit == 'acre':
        return value * 43560
    elif unit == 'sqyrd':
        return value * 9
    elif unit == 'sqin':
        return value / 144
    else:
        return value

df['super_area_sqft'] = df['super area'].apply(parse_area)

df['carpet_area_sqft'] = df['carpet area'].apply(parse_area)

print(df['super area'].isna().sum())
print(df['carpet area'].isna().sum())

print(df[df['super_area_missing'] == 0].groupby('super_area_sqft')['amount(in rupees)'].agg(['mean', 'count']))

print(df[df['carpet_area_missing'] == 0].groupby('carpet_area_sqft')['amount(in rupees)'].agg(['mean', 'count']))

print(df['carpet_area_sqft'].isna().sum())

print(df.head(10))

df = df[
    (df['carpet_area_sqft'].between(100, 20000)) |
    (df['carpet_area_sqft'].isna())
]
df = df[
    (df['super_area_sqft'].between(100, 5000)) |
    (df['super_area_sqft'].isna())
]

```

```

]

# ===== 9. Facing и сырые столбцы =====
# Вывод: facing.fillna('missing'); сырые площади и car parking больше не нужны.

print(df['facing'].unique())

print(df.columns)

df = df.drop(['super area', 'carpet area', 'super_area_missing', 'carpet_area_missing'], axis=1)

df = df.drop('car parking', axis=1)

df['facing'] = df['facing'].fillna('missing')
df['facing'] = df['facing'].str.lower()

print(df['facing'].unique())

# ===== 10. Парсинг этажа =====
# Вывод: «N out of M» → floor_current и floor_total, Ground=0. Битый формат без total отбрасывается.

def parse_floor_current(x):
    if pd.isna(x):
        return np.nan

    x = str(x).strip().lower()

    if x.startswith('ground'):
        return 0

    match = re.search(r'^(\d+)', x)
    if match:
        return int(match.group(1))

    return np.nan

def parse_floor_total(x):
    if pd.isna(x):
        return np.nan

    x = str(x).strip().lower()

    match = re.search(r'out of\s*(\d+)', x)
    if match:
        return int(match.group(1))

    match = re.search(r'^(\d+)\$', x)
    if match:
        return int(match.group(1))

    return np.nan

df['floor_current'] = df['floor'].apply(parse_floor_current)
df['floor_total'] = df['floor'].apply(parse_floor_total)

mask = df['floor'].notna() & df['floor_total'].isna()

df = df[~mask]

print(df[['floor', 'floor_current', 'floor_total']].head(5))

df = df.drop('floor', axis=1)

print(df[target].head(5))

print(df.columns)

```

Гипотеза подтвердилась частично, мебелированная на 11% чаще resale, чем new. resale дешевле на 12% в сравнении с new, что значимо. Полу-мебелированная на 4.2% дороже чем мебелированная. В средних значениях имеем полу-мебелированная на 300 рублей дороже. А resale на 850 рублей дешевле. Разница в использовании 11% следовательно имеем $11/100 * 850 = 90$. Итого $300+90$ имеем почти 400 рублей. То есть разница в цене resale не перебивает разницу в мебелировании.

```
#

#sns.heatmap(df.select_dtypes(include='number').corr())

# ===== 11. Пайплайн RF и log-таплет =====
# Вывод: split 80/20, log1p(amount). Финальная модель — RF из Optuna (n_estimators=180,
max_depth=25).
# Test RF: MAE 799 909, R² 0.9414, MAPE 8.93%.

from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, r2_score, mean_squared_error
import time
from sklearn.metrics import mean_absolute_percentage_error
import optuna
import lightgbm as lgb
import category_encoders as ce
from catboost import CatBoostRegressor

#BASELINE

X = df.drop(target, axis=1)
y = df[target]

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"\nTapret:")
print(y.describe())

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# encoder_facing = OneHotEncoder(handle_unknown='ignore', drop='first', sparse_output=False)

# facing_train = encoder_facing.fit_transform(X_train[['facing']])
# facing_test = encoder_facing.transform(X_test[['facing']])

# facing_cols = encoder_facing.get_feature_names_out(['facing'])
# facing_train_df = pd.DataFrame(facing_train, columns=facing_cols, index=X_train.index)
# facing_test_df = pd.DataFrame(facing_test, columns=facing_cols, index=X_test.index)

# X_train = X_train.drop(columns = 'facing')
# X_test = X_test.drop(columns = 'facing')

# X_train = pd.concat([X_train, facing_train_df], axis=1)
# X_test = pd.concat([X_test, facing_test_df], axis=1)

# encoder = ce.TargetEncoder(cols=['location'], smoothing=10)

# X_train = encoder.fit_transform(X_train, y_train)
# X_test = encoder.transform(X_test)

y_train_log = np.log1p(y_train)
y_test_log = np.log1p(y_test)

numeric_features = X_train.select_dtypes(include='number').columns.tolist()
categorical_features = X_train.select_dtypes(include='object').columns.tolist()

#PIPELINE Random Forest Regressor
#numeric_transformer = Pipeline([
```

```

# ('imputer', SimpleImputer(strategy='median')),
# ('scaler', StandardScaler())
# ])

# categorical_transformer = Pipeline([
#     ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
#     ('onehotencode', OneHotEncoder(handle_unknown='ignore', drop='first', max_categories=20))
# ])
# preprocessor = ColumnTransformer([
#     ('num', numeric_transformer, numeric_features),
#     ('cat', categorical_transformer, categorical_features)
# ])
# model = RandomForestRegressor(n_estimators= 180, max_depth=25, min_samples_split=2,
min_samples_leaf=1, max_features=0.7509776412241582, max_samples=0.9562634426883665, n_jobs=1,
random_state=42)

# pipeline = Pipeline([
#     ('preprocessor', preprocessor),
#     ('model', model)
# ])

# pipeline.fit(X_train, y_train_log)

print(f"X_train shape: {X_train.shape}")
print(f"y_train_log shape: {y_train_log.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_test_log shape: {y_test_log.shape}")

# ===== 12. Сравнение LightGBM и CatBoost =====
# Вывод: LGBM после Optuna MAPE 9.39% (MAE 841 тыс.), CatBoost MAPE 10.29% (MAE 929 тыс.).
# Оба хуже RF по MAE/MAPE; в отчёт как итоговая модель взят Random Forest.

# # PIPELINE LGBM regressor
# numeric_transformer = Pipeline([
#     ('imputer', SimpleImputer(strategy='median')),
#     ('scaler', StandardScaler())
# ])

# categorical_transformer = Pipeline([
#     ('imputer', SimpleImputer(strategy='constant', fill_value='missing'))
# ])
# preprocessor = ColumnTransformer([
#     ('num', numeric_transformer, numeric_features),
#     ('cat', categorical_transformer, categorical_features)
# ])

# model = lgb.LGBMRegressor(n_estimators = 1971, learning_rate = 0.08855706462133571, num_leaves =
136, max_depth = 10, min_child_samples = 46, subsample= 0.8372315466266708, colsample_bytree =
0.8333324377961918, reg_lambda = 0.27299801033544263)

# pipeline = Pipeline([
#     ('preprocessor', preprocessor),
#     ('model', model)
# ])

# pipeline.fit(
#     X_train, y_train_log
# )

# PIPELINE cat GBM regressor

cat_features = ['location', 'facing']

X_train_cat = X_train.copy()

```

```

X_test_cat = X_test.copy()

for col in cat_features:
    X_train_cat[col] = X_train_cat[col].astype(str)
    X_test_cat[col] = X_test_cat[col].astype(str)

from sklearn.impute import SimpleImputer

numeric_cols = [col for col in X_train_cat.columns if col not in cat_features]
imputer = SimpleImputer(strategy='median')

X_train_cat[numeric_cols] = imputer.fit_transform(X_train_cat[numeric_cols])
X_test_cat[numeric_cols] = imputer.transform(X_test_cat[numeric_cols])

# model_cat = CatBoostRegressor(
#     iterations=2000,
#     learning_rate=0.05,
#     depth=8,
#     cat_features=cat_features,
#     verbose=100,
#     random_seed=42,
#     early_stopping_rounds=100
# )
# model_cat.fit(
#     X_train_cat, y_train_log,
#     eval_set=(X_test_cat, y_test_log),
#     use_best_model=True
# )

y_pred_log_cat = model_cat.predict(X_test_cat)
y_pred_cat = np.expml(y_pred_log_cat)

mae_cat = mean_absolute_error(y_test, y_pred_cat)
rmse_cat = np.sqrt(((y_test - y_pred_cat) ** 2).mean())
r2_cat = r2_score(y_test, y_pred_cat)
mape_cat = mean_absolute_percentage_error(y_test, y_pred_cat)

print(f"\n{'=' * 60}")
print(f"{'CatBoost Regressor'}")
print(f"{'=' * 60}")
print(f"MAE: {mae_cat:,.0f} рупий")
print(f"RMSE: {rmse_cat:,.0f} рупий")
print(f"R²: {r2_cat:.4f}")
print(f"MAPE: {mape_cat * 100:.2f}%")

y_pred_log = pipeline.predict(X_test)
y_pred = np.expml(y_pred_log)

mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(((y_test - y_pred) ** 2).mean())
r2 = r2_score(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)

print(f"\n{'=' * 60}")
print(f"{'LGBM Regressor'}")
print(f"{'=' * 60}")
print(f"MAE: {mae:,.0f} рупий")
print(f"RMSE: {rmse:,.0f} рупий")
print(f"R²: {r2:.4f}")
print(f"MAPE: {mape * 100:.2f}%")

# ===== 13. Optuna objective для RF / LGBM / CatBoost =====
# Вывод: RF — 50 trial; LGBM — 20 trial (лучший MAPE 9.36%). Objective CatBoost в ячейке черновик,
# не гонялся.

```

```

def objective(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 50, 300),
        'max_depth': trial.suggest_int('max_depth', 5, 30),
        'min_samples_split': trial.suggest_int('min_samples_split', 2, 20),
        'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 10),
        'max_features': trial.suggest_float('max_features', 0.3, 1.0),
        'max_samples': trial.suggest_float('max_samples', 0.5, 1.0)
    }

    model = RandomForestRegressor(**params, n_jobs=1, random_state=42)
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    pipeline.fit(X_train, y_train_log)

    y_pred_log = pipeline.predict(X_test)
    y_pred = np.expml(y_pred_log)

    mape = mean_absolute_percentage_error(y_test, y_pred)
    return mape

def objective_lgb(trial):
    params = {
        'n_estimators': trial.suggest_int('n_estimators', 1000, 3000),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'num_leaves': trial.suggest_int('num_leaves', 31, 256),
        'max_depth': trial.suggest_int('max_depth', 6, 12),
        'min_child_samples': trial.suggest_int('min_child_samples', 5, 100),
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-2, 10, log=True)
    }

    model = lgb.LGBMRegressor(**params, n_jobs=1, random_state=42, verbose=-1)
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

    pipeline.fit(X_train, y_train_log)

    y_pred_log = pipeline.predict(X_test)
    y_pred = np.expml(y_pred_log)

    mape = mean_absolute_percentage_error(y_test, y_pred)
    return mape

def objective_cgb(trial):
    params = {
        'iterations': trial.suggest_int('n_estimators', 1000, 3000),
        'learning_rate': trial.suggest_float('learning_rate', 0.01, 0.1),
        'depth': trial.suggest_int('max_depth', 6, 16),
        'cat_features': cat_features
        'subsample': trial.suggest_float('subsample', 0.6, 1.0),
        'colsample_bytree': trial.suggest_float('colsample_bytree', 0.5, 1.0),
        'reg_lambda': trial.suggest_float('reg_lambda', 1e-2, 10, log=True)
    }

    model = lgb.LGBMRegressor(**params, n_jobs=1, random_state=42, verbose=-1)
    pipeline = Pipeline([
        ('preprocessor', preprocessor),
        ('model', model)
    ])

```

```

pipeline.fit(X_train, y_train_log)

y_pred_log = pipeline.predict(X_test)
y_pred = np.expml(y_pred_log)

mape = mean_absolute_percentage_error(y_test, y_pred)
return mape

# model_cat = CatBoostRegressor(
#     iterations=2000,
#     learning_rate=0.05,
#     depth=8,
#     cat_features=cat_features,
#     verbose=100,
#     random_seed=42,
#     early_stopping_rounds=100
# )

#study = optuna.create_study(direction='minimize')
#study.optimize(objective, n_trials=50, show_progress_bar=False)

#print(f"Best MAPE: {study.best_value * 100:.2f}%")
#print(f"Best params: {study.best_params}")

# study = optuna.create_study(direction='minimize')
# study.optimize(objective_lgb, n_trials=20, show_progress_bar=False)

# print(f"Best MAPE: {study.best_value * 100:.2f}%")
# print(f"Best params: {study.best_params}")

```